

Department of Computer Science
Banasthali Vidhyapith



**Software Requirement Specifications (SRS) and
Software Design Specifications (SDS)**
[Eduvision-Student performance prediction platform]
[G-16]

Guided By:-

Dr.Manoj Kumar Sain

Submitted By:-

Nandini Pandey(251H0660043)

Tushti Savarn(251H0660066)

BarkhaPathak(251H0660018)

[MCA 2nd Semester]

Chapter 1 SRS

TABLE OF CONTENT

1. Introduction

1.1 Purpose.....	4
1.2 Scope.....	4
1.3 Overview.....	5

2. General Description

2.1 Product Perspective	
2.1.1 Product Function	5
2.1.2 Hardware Interface.....	6
2.1.3 Software Interface.....	6
2.1.4 Communication Interface.....	7
2.2 User Characteristics.....	7
2.3 General Constraints.....	8
2.4 Technologies used.....	8

3. Specific Requirement

3.1 Functional Requirements.....	9
3.2 Non-Functional Requirements.....	10
3.2.1 Performance.....	10
3.2.2 Security.....	11
3.2.3 Scalability.....	11
3.2.4 Usability.....	11
3.2.5 Maintainability.....	12
3.2.6 Reliability.....	12

1.Introduction

1.1 Purpose of the Project

The purpose of the *EduVision – Student Performance Prediction Platform* is to design and develop a web-based system that analyzes students' academic data and predicts their future performance using data-driven techniques. In many traditional academic systems, student information such as attendance and marks is recorded but not effectively analyzed to support early identification of academic risks. EduVision aims to overcome this limitation by providing predictive insights that help improve academic outcomes.

The platform is designed to support students by helping them understand their academic performance, identify weak areas, and receive guidance for improvement. Teachers can monitor student progress, analyze class performance, and identify at-risk students at an early stage. Administrators can oversee academic trends and manage the overall system efficiently. EduVision also integrates a basic chatbot feature to assist users with academic queries, making the system interactive and user-friendly.

1.2 Scope of the Project

The scope of the EduVision project is limited to academic performance monitoring and prediction within an educational institution. The system supports three types of users: Students, Teachers, and Administrators, each with defined roles and access permissions.

The project includes functionalities such as user authentication, attendance management, marks management, performance visualization, and machine-learning-based prediction of student performance. A basic chatbot module is included to answer common academic questions and provide study-related guidance. The platform also offers limited placement-related insights to help students understand their readiness for future career opportunities.

The system is implemented as a web-based application using beginner-friendly technologies and is intended for academic and educational use. Advanced enterprise-level features, large-scale deployment, and real-time integrations are outside the scope of this project.

1.3 Overview of the Project

EduVision is an intelligent academic analytics platform that collects and processes student-related data such as attendance records, examination marks, and previous academic performance. Using this data, the system applies predictive techniques to estimate future academic performance and classify students based on risk levels.

The platform provides separate dashboards for students, teachers, and administrators. Students can view their academic details, prediction results, and recommendations. Teachers can upload attendance and marks, analyze subject-wise and class-wise performance, and identify students who require additional academic support. Administrators manage users, academic structures, and overall system analytics.

By integrating web technologies with basic machine learning concepts, EduVision demonstrates how data-driven systems can enhance academic decision-making for students.

2.1 Product Perspective

EduVision is a web-based academic analytics and student performance prediction platform designed to support educational institutions. The system functions as an independent application that manages academic data, analyzes student performance, and provides predictive insights using machine learning techniques. It integrates student records, teacher inputs, and administrative controls into a single centralized system.

The platform follows a role-based access model where students, teachers, and administrators interact with the system according to their assigned roles. EduVision aims to enhance the traditional academic management system by adding intelligent prediction, visualization, and guidance features.

2.1.1 Product Functions

The major functions of EduVision include:

1. User registration and secure login for students, teachers, and administrators
2. Attendance management and monitoring
3. Marks and academic record management
4. Performance visualization using graphs and dashboards

5. Prediction of student performance using historical academic data
6. Identification of at-risk students based on prediction results
7. Basic chatbot support for academic queries and guidance
8. Administrative management of users, departments, and academic data

2.1.2 Hardware Interface

EduVision is a web-based application and does not require any specialized hardware. The minimum hardware requirements are:

1. A computer, laptop, tablet, or smartphone
2. Processor: Dual-core or higher
3. RAM: Minimum 4 GB
4. Internet connection for accessing the web application

The system runs entirely through a web browser and does not require additional hardware peripherals.

2.1.3 Software Interface

The software interfaces used in EduVision include:

1. Web Browser: Google Chrome, Mozilla Firefox, Microsoft Edge, or any modern browser
2. Operating System: Windows
3. Backend Framework: Django (Python)

4. Frontend Technologies: HTML, Tailwind CSS, JavaScript
5. Database: SQLite

The interaction between the frontend and backend is handled through server-side rendering using Django templates.

2.1.4 Communication Interface

EduVision uses standard internet communication protocols for data exchange:

1. HTTP/HTTPS for client-server communication
2. TCP/IP for network connectivity
3. Secure authentication mechanisms for user login

All interactions between users and the system occur through a web interface using a client-server architecture.

2.2 User Characteristics

The users of EduVision are categorized into three groups:

1. Students: Basic computer literacy and ability to use a web browser. Students interact with the system to view attendance, marks, predictions, and academic guidance.
2. Teachers: Familiarity with academic data entry and basic web applications. Teachers upload attendance, marks, and monitor student performance.
3. Administrators: Users with technical and managerial understanding who manage users, departments, and system-level data.

No advanced technical knowledge is required for any user role.

2.3 General Constraints

The system operates under the following constraints:

1. Internet connectivity is required to access the platform
2. Prediction accuracy depends on the quality and availability of academic data
3. The system is designed for small to medium-scale academic institutions
4. Limited to academic and educational use
5. Advanced real-time analytics and large-scale deployment are not supported

2.4 Technologies Used

The technologies used for developing EduVision are:

1. Frontend: HTML, Tailwind CSS, JavaScript
2. Backend: Django (Python)
3. Database: SQLite
4. Machine Learning: Python (basic prediction models)
5. Tools: VS Code, Git, Web Browser

3. Specific Requirement

3.1 Functional Requirements

3.1.1 User Registration

1. The system shall allow new students to register
2. The system shall collect details such as name, email, roll number, and password.

3. The system shall validate input data before creating an account.

3.1.2 User Login

1. The system shall allow registered users to log in using username and password.
2. The system shall authenticate user credentials
3. The system shall display an error message for invalid login attempts.

3.1.3 Role-Based Access

The system shall support different user roles:

1. Admin
2. Teacher
3. Student

The system shall restrict access based on user role

3.1.4 . Upload Academic Data

1. The system shall allow teachers to upload student marks.
2. The system shall allow uploading attendance data.

3.1.5 View Academic Records

1. The system shall allow students to view their marks.
2. The system shall allow students to view attendance records.
3. Students shall only access their own data

3.1.6. Performance Prediction

1. The system shall accept academic inputs (marks, attendance, etc.).
2. The system shall process input data using a prediction algorithm.
3. The system shall generate predicted performance results.
4. The system shall display prediction results to the student.

3.1.7. Chat Feature

1. The system shall allow students to communicate with teachers/admin.
2. The system shall store chat history.
3. The system shall notify users of new messages.

3.1.8. Logout

1. The system shall allow users to securely log out.
2. The system shall terminate the session after logout.

3.1.9 Report Generation

1. The system shall generate performance reports.

3.2 NON-FUNCTIONAL REQUIREMENTS

3.2.1 Performance

1. The system shall provide fast, reliable, and efficient responses to user requests.
2. The dashboard shall load within 3–5 seconds under normal internet conditions.
3. Prediction results shall be generated within 2–4 seconds after submission of input data.
4. The system shall support at least 200 concurrent users without noticeable performance degradation.
5. Database queries shall be optimized to ensure fast retrieval and storage of student records.
6. The system shall maintain a minimum uptime of 95%. Performance shall remain stable during peak usage periods such as examination and result declaration times.
7. Server response time shall not exceed acceptable thresholds under normal workload conditions

3.2.2 Security

1. The system shall ensure the confidentiality, integrity, and protection of student data.
2. All users shall authenticate using secure login credentials (email and password).

3. Passwords shall be stored in encrypted/hashed format in the database. Role-Based Access Control (RBAC) shall restrict access based on user roles (Admin, Teacher, Student).
4. Only authorized users (Teacher) shall modify student records.
5. Students shall only access their own academic and prediction data.
6. All data transmission shall occur over HTTPS protocol. The system shall maintain logs of important activities for auditing and monitoring.
7. The system shall comply with applicable data privacy and protection standards

3.2.3 Scalability

1. The system shall be capable of handling growth in users, data, and functionality.
2. The system shall support expansion from a single institution to multiple institutions.
3. The database design shall accommodate large volumes of student records efficiently.
4. The architecture shall allow horizontal and vertical scaling.
5. The system shall support cloud deployment for improved scalability and availability.
6. Additional modules such as attendance tracking, analytics, and AI-based recommendations shall be integrated in the future without major architectural changes.
7. Performance shall remain consistent as user traffic increases.

3.2.4. Usability

1. The system shall provide an intuitive and user-friendly experience.
2. The interface shall be simple, clear, and easy to understand.
3. Navigation shall be intuitive with clearly labeled menus and buttons.
4. Error messages shall be meaningful and guide users to resolve issues.
5. The system shall support responsive design across mobile, tablet, and desktop devices.
6. Users shall require minimal training to operate the system effectively.
7. The design shall prioritize clarity and simplicity to enhance user experience.

3.2.5 Maintainability

1. The system shall be designed using modular architecture for easier updates.
2. The system shall allow easy modification of prediction algorithms.
3. The source code shall be properly documented.
4. Future enhancements such as additional analytics modules can be integrated without affecting existing functionality.

3.2.6 Reliability

1. The system shall perform prediction calculations accurately based on provided input data.
2. The system shall ensure data consistency and prevent data loss.
3. The system shall handle invalid or incorrect inputs gracefully.
4. The system shall operate without crashing under normal usage conditions.

Chapter 2 SDS

TABLE OF CONTENTS

1. Introduction

1.1 Purpose.....	15
1.2 Scope.....	15
1.3 Terms, Definitions, Acronyms and Abbreviations.....	15
1.4 Overview of Document	16

2. System Architectural Design

2.1 High Level Design Overview.....	17
2.2 Design Diagrams	
2.2.1 Use Case Diagram.....	18
2.2.2 Activity Diagram.....	21
2.2.3 Sequence Diagram.....	25
2.2.4 ER Diagram.....	27

3. TYPES OF TESTS (With Implementation).....28

4. References29

1. Introduction

1.1 Purpose

The purpose of this Software Design Specification (SDS) document is to describe the overall design and architecture of the Smart Student Performance Prediction System. This document provides a detailed explanation of system components, design decisions, data flow, interfaces, and operational behavior of the system.

The SDS serves as a reference for developers, project evaluators, and stakeholders to understand how the system is structured and how it will function. It ensures that the system design meets the requirements specified in the Software Requirement Specification (SRS) document.

1.2 Scope

The Smart Student Performance Prediction System is a web-based application designed to analyze student academic data such as marks, attendance, and performance records to predict future academic performance using data analysis techniques.

The system allows:

1. Admin to manage users and database records.
2. Teachers upload student marks and attendance.
3. Students can view their attendance, marks, and performance prediction.
4. The system generates performance predictions based on stored data.

This project aims to assist educational institutions in identifying weak-performing students early so that corrective measures can be taken to improve academic results.

1.3 Terms, Definitions, Acronyms and Abbreviations

1. SRS – Software Requirement Specification
2. SDS – Software Design Specification
3. DFD – Data Flow Diagram

4. UML – Unified Modeling Language
5. Admin – System administrator who manages users and database
6. Prediction Model – Algorithm used to analyze student data and predict performance
7. Database – Structured collection of student records

1.4 Overview of Document

This Software Design Specification (SDS) document provides a detailed description of the design and structure of the Smart Student Performance Prediction System.

The document explains the overall architecture of the system and describes how different modules interact with each other.

This document includes:

1. System architecture and overall design structure
2. Use Case Diagrams representing interactions between users and the system
3. Activity Diagrams showing workflow of system processes
4. Sequence Diagrams illustrating step-by-step interaction between system components
5. ER Diagram representing the database structure and relationships between entities
6. Database design and module descriptions

These design models help in understanding how the system will be implemented and how different components of the system work together.

2. System Architectural Design

2.1 High Level Design Overview

The Smart Student Performance Prediction System is designed using a layered architecture to ensure modularity, scalability, and maintainability. The system consists of three major layers:

1. Presentation Layer

This layer provides the user interface for Admin, Teacher, and Student. Users interact with the system through web-based forms and dashboards.

2. Application Layer

This layer contains the business logic of the system. It processes user requests such as login authentication, uploading marks, managing attendance, and generating performance predictions.

3. Data Layer

This layer manages the database. It stores student records, marks, attendance data, user credentials, and prediction results.

The layered architecture ensures that each component functions independently while maintaining smooth communication between layers.

2.2 Design Diagrams

2.2.1 Use Case Diagram

The Use Case Diagram represents the interaction between users and the system. The system consists of three main actors:

1. Admin
2. Teacher
3. Student

Each actor interacts with the system based on their role.

1. The Admin manages users and database operations.
2. The Teacher uploads marks and attendance and views predictions.
3. The Student views attendance, marks, prediction results, and chat.

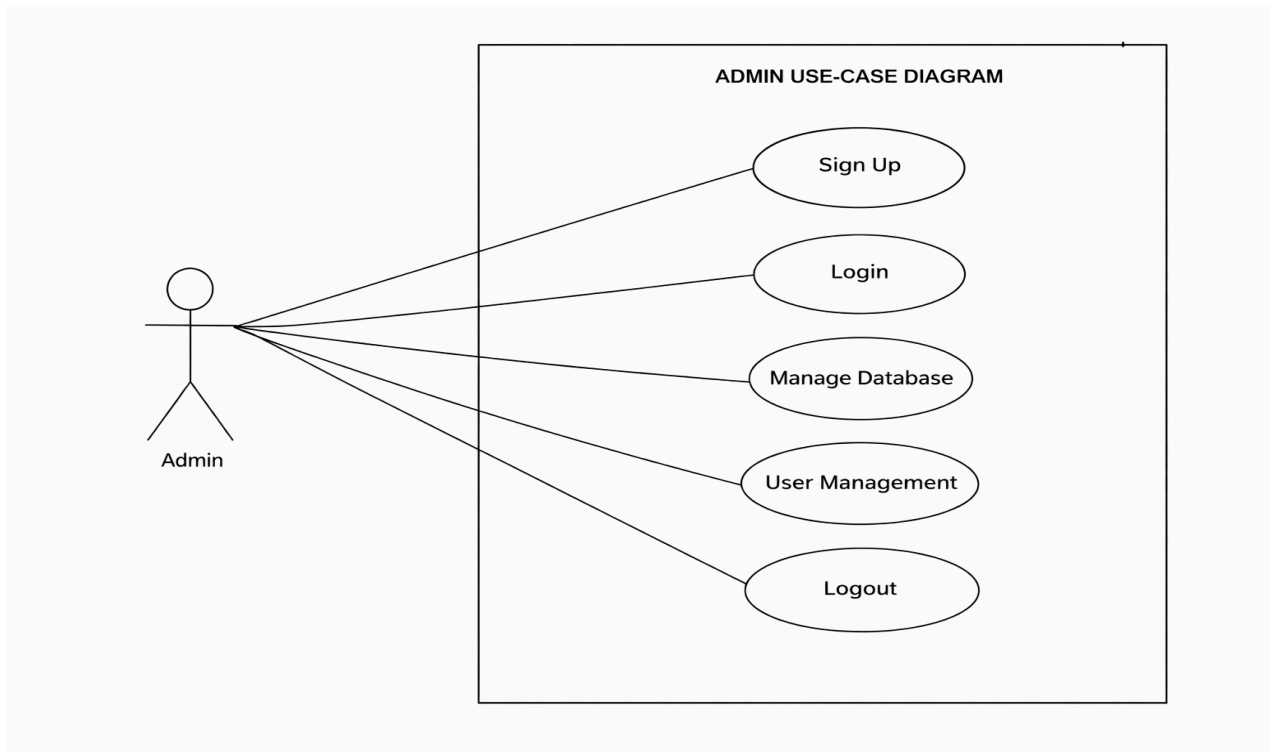
The Use Case Diagram helps in understanding the functional requirements of the system.

Admin Use Case Diagram Description

The Admin Use Case Diagram represents the functionalities performed by the Admin in the Smart Student Performance Prediction System. The Admin is responsible for managing the overall system and controlling user activities.

The Admin can perform the following operations:

1. Signup – Create an admin account.
2. Login – Access the system using valid credentials.
3. Manage Database – Maintain and update system records.
4. User Management – Add, update, or remove student and teacher accounts.
5. Logout – Exit securely from the system.

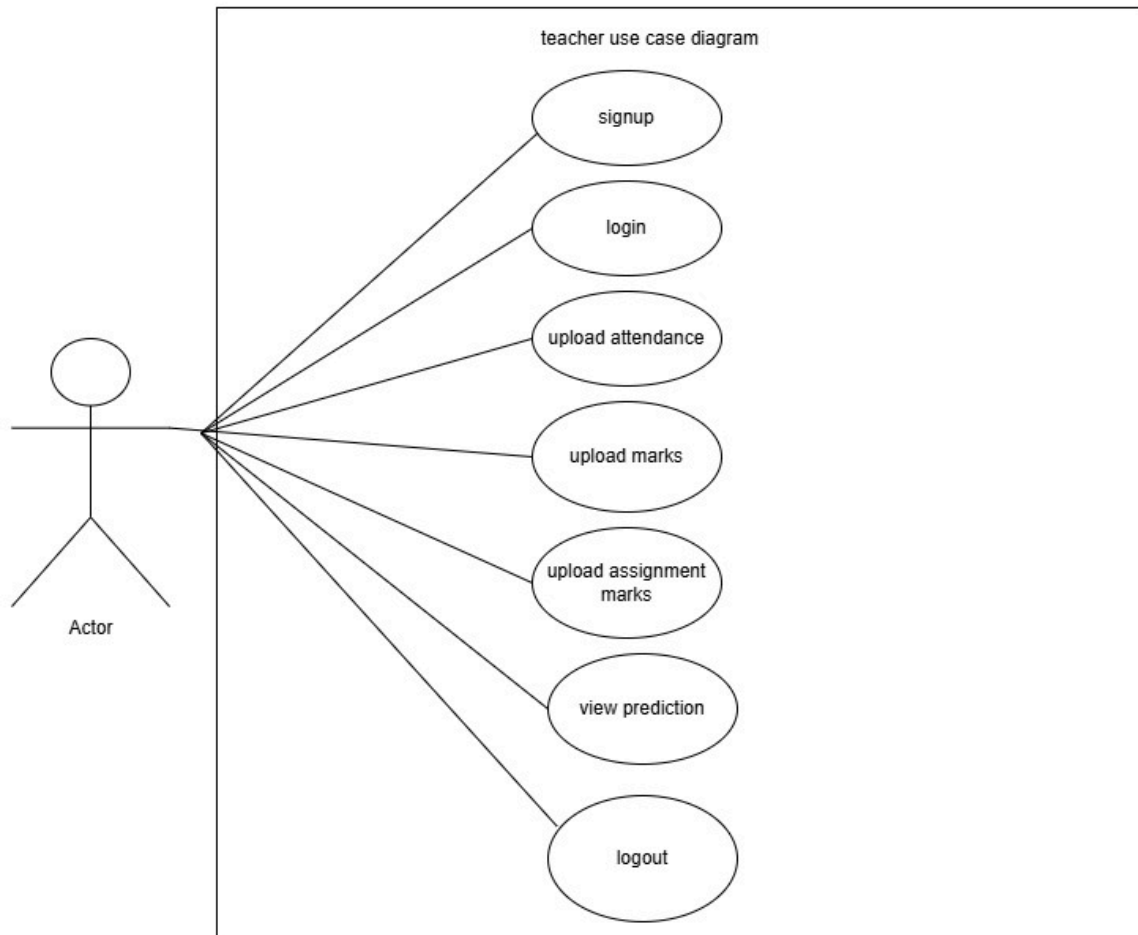


Teacher Use Case Diagram Description

The Teacher Use Case Diagram represents the functionalities performed by the Teacher in the Smart Student Performance Prediction System. The Teacher plays an important role in maintaining student academic records and supporting the prediction process.

The Teacher can perform the following operations:

1. Signup – Create a teacher account in the system.
2. Login – Access the system using valid credentials.
3. Upload Marks – Enter and update student academic marks.
4. Upload Attendance – Record and update student attendance details.
5. View Prediction – View the performance prediction results of students.
6. Logout – Securely exit from the system.

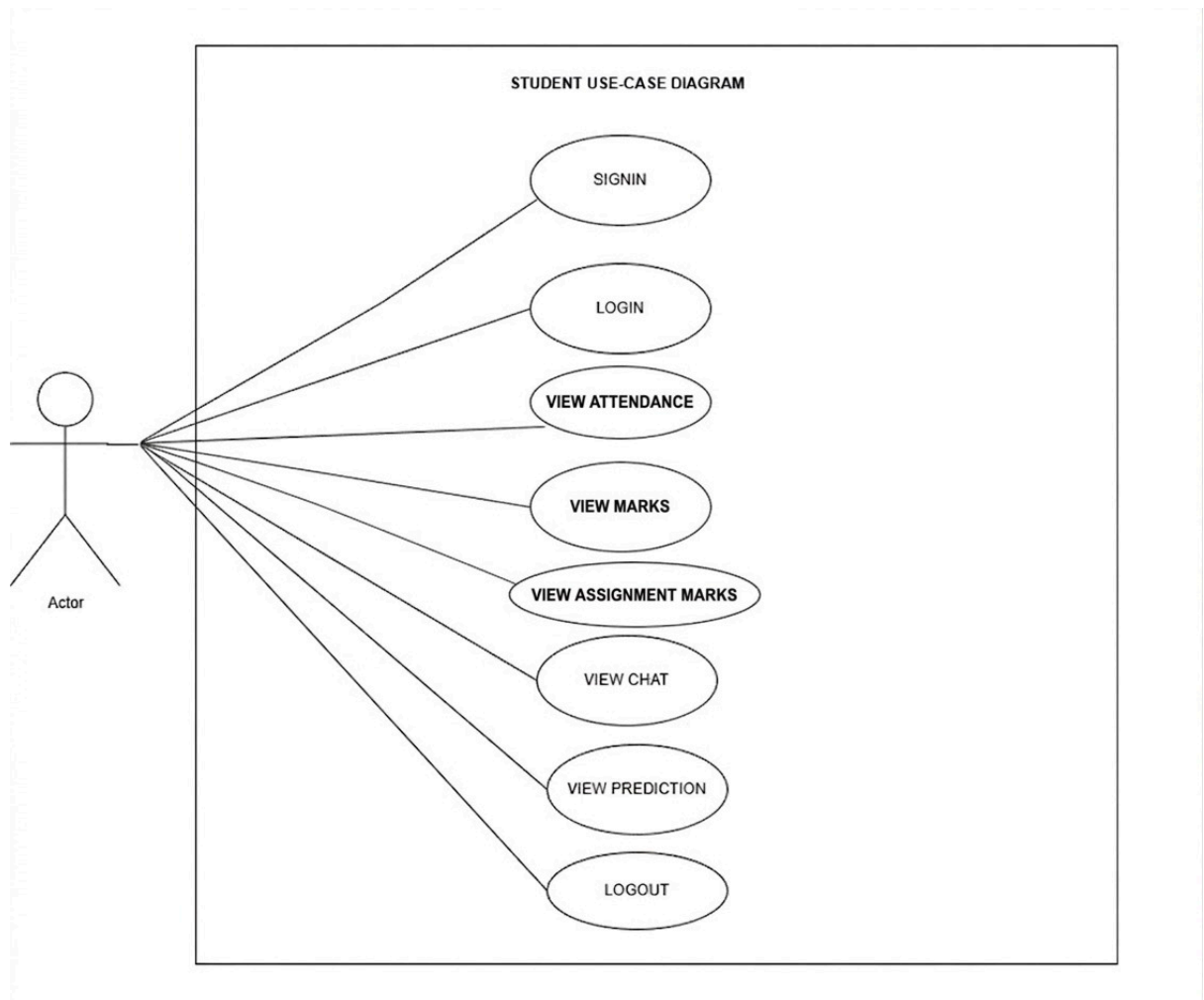


Student Use Case Diagram Description

The Student Use Case Diagram represents the functionalities available to students in the Smart Student Performance Prediction System. The system allows students to access their academic information and view performance predictions.

The Student can perform the following operations:

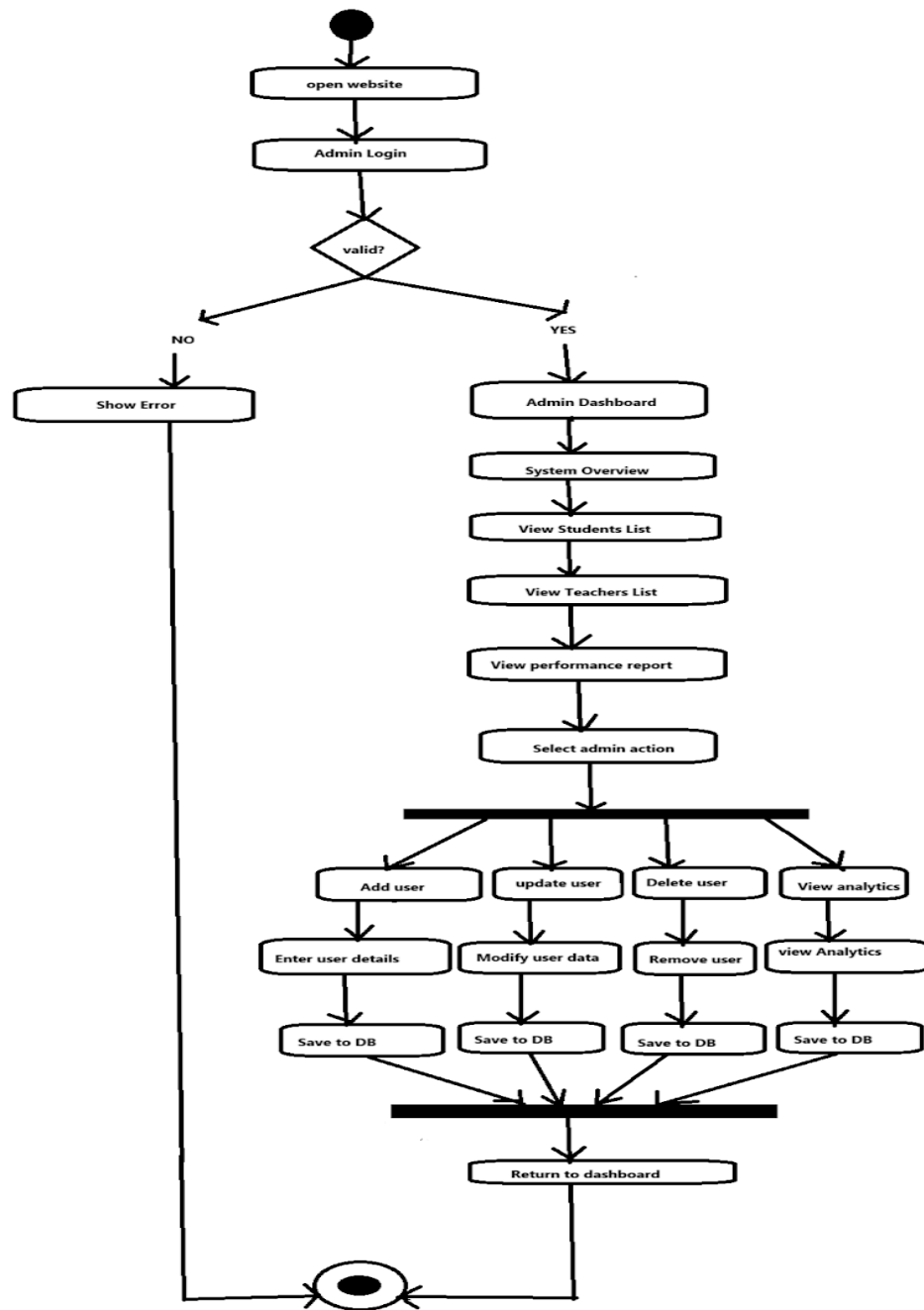
1. Signup – Create a student account in the system.
2. Login – Access the system using valid credentials.
3. View Attendance – Check attendance records uploaded by teachers.
4. View Marks – View academic marks and subject-wise performance.
5. View Chat – Communicate with teachers through the chat feature.
6. View Prediction – View predicted performance results generated by the system.
7. Logout – Securely exit from the system.



2.2.2 Activity Diagram

Admin Activity Diagram Description

The Admin Activity Diagram represents the workflow of activities performed by the Admin in the Smart Student Performance Prediction System. The process begins when the Admin opens the system and logs in using valid credentials. After successful authentication, the Admin accesses the dashboard. From the dashboard, the Admin can manage users and maintain the system database.



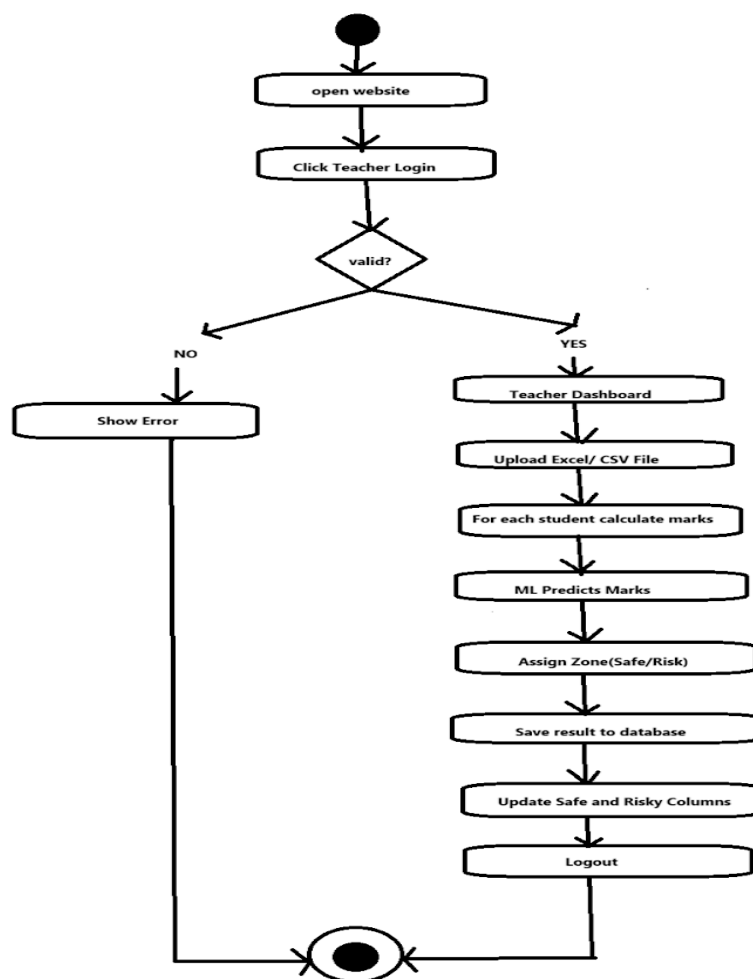
Teacher Activity Diagram Description

The Teacher Activity Diagram shows the sequence of actions performed by the Teacher in the system.

The process starts when the Teacher logs in to the system. After successful login, the Teacher accesses the dashboard. The Teacher can upload student marks and attendance details.

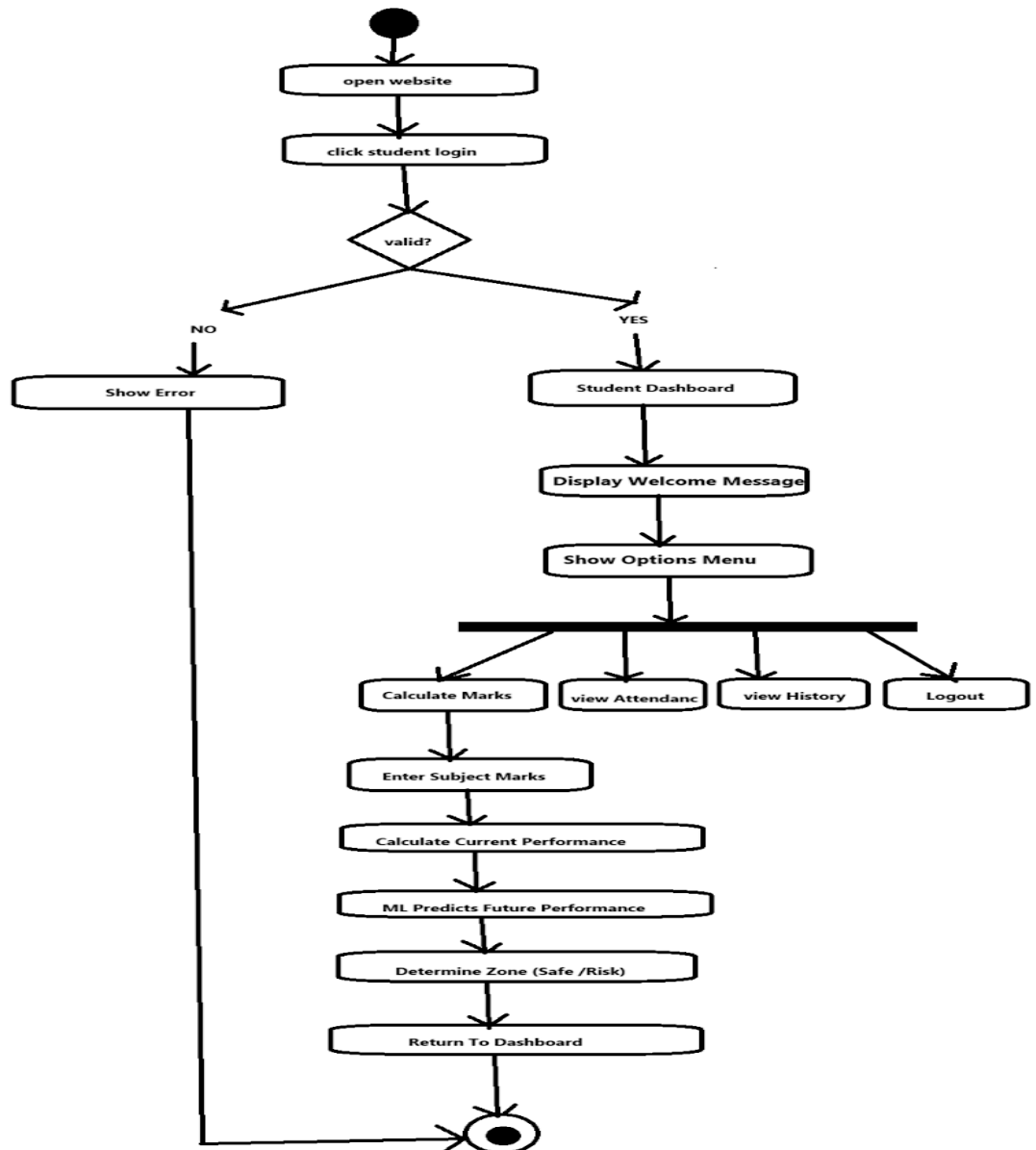
Once the data is submitted, the system stores it in the database and updates records. The Teacher can also view student performance predictions. After completing the tasks, the Teacher logs out of the system.

This diagram represents the workflow of academic data management.



Student Activity Diagram Description

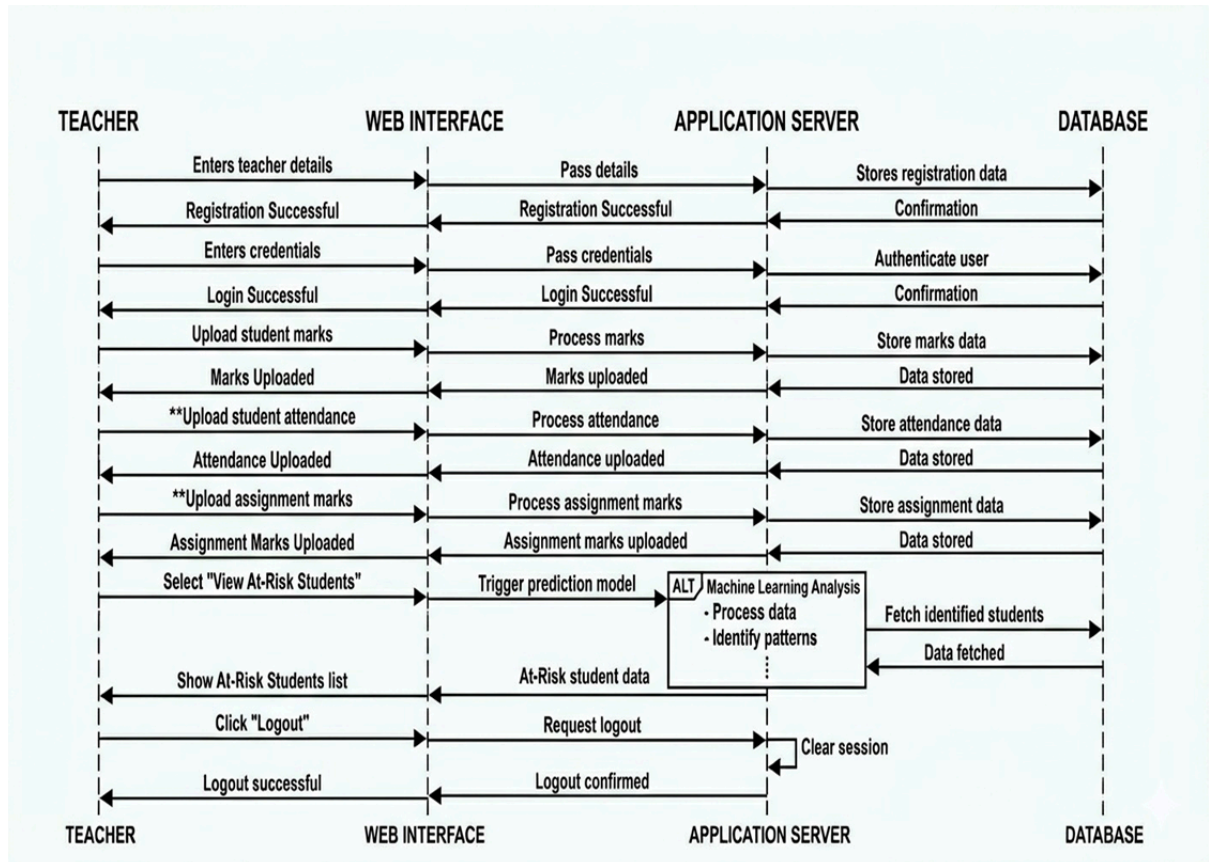
The Student Activity Diagram represents the workflow followed by a student while using the system. The process begins with student login. After successful authentication, the student accesses the dashboard. The student can view attendance records, view marks, check performance prediction results, and use the chat feature. The system retrieves the required data from the database and displays it to the student. After viewing the information, the student logs out of the system.



2.2.3 Sequence Diagram

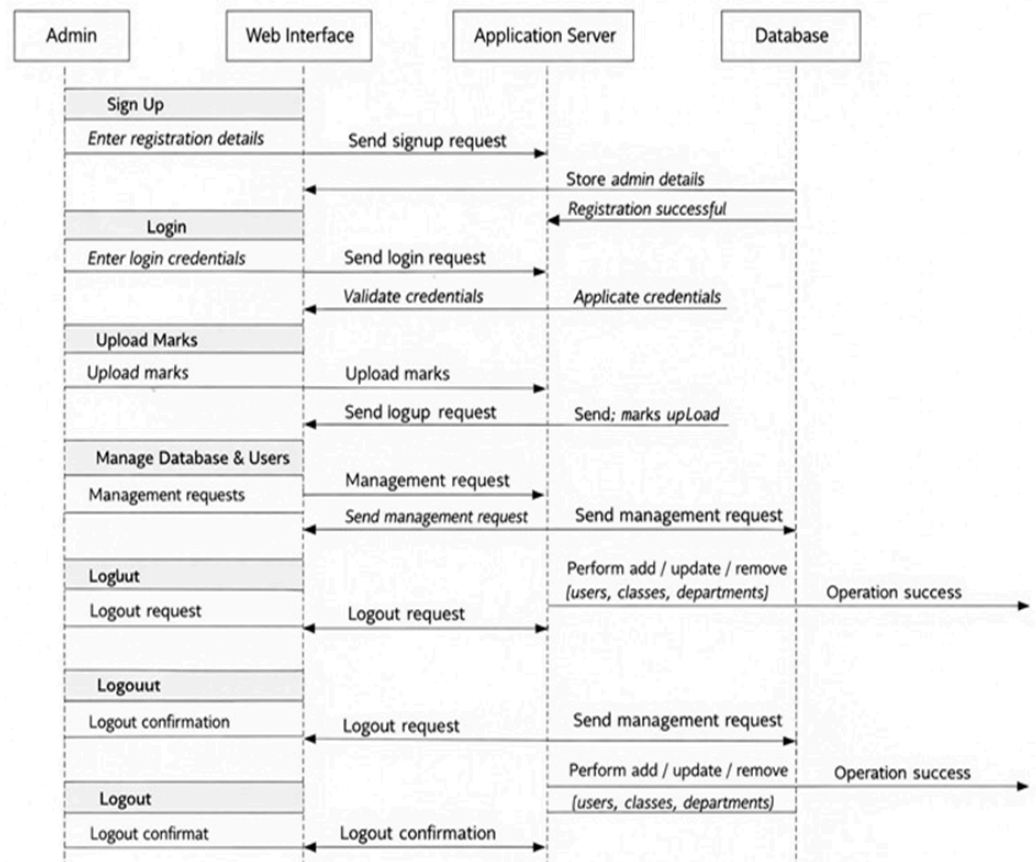
Teacher Sequence Diagram – Description

The Teacher Sequence Diagram represents interactions between Teacher, Web Interface, Application Server, and Database.



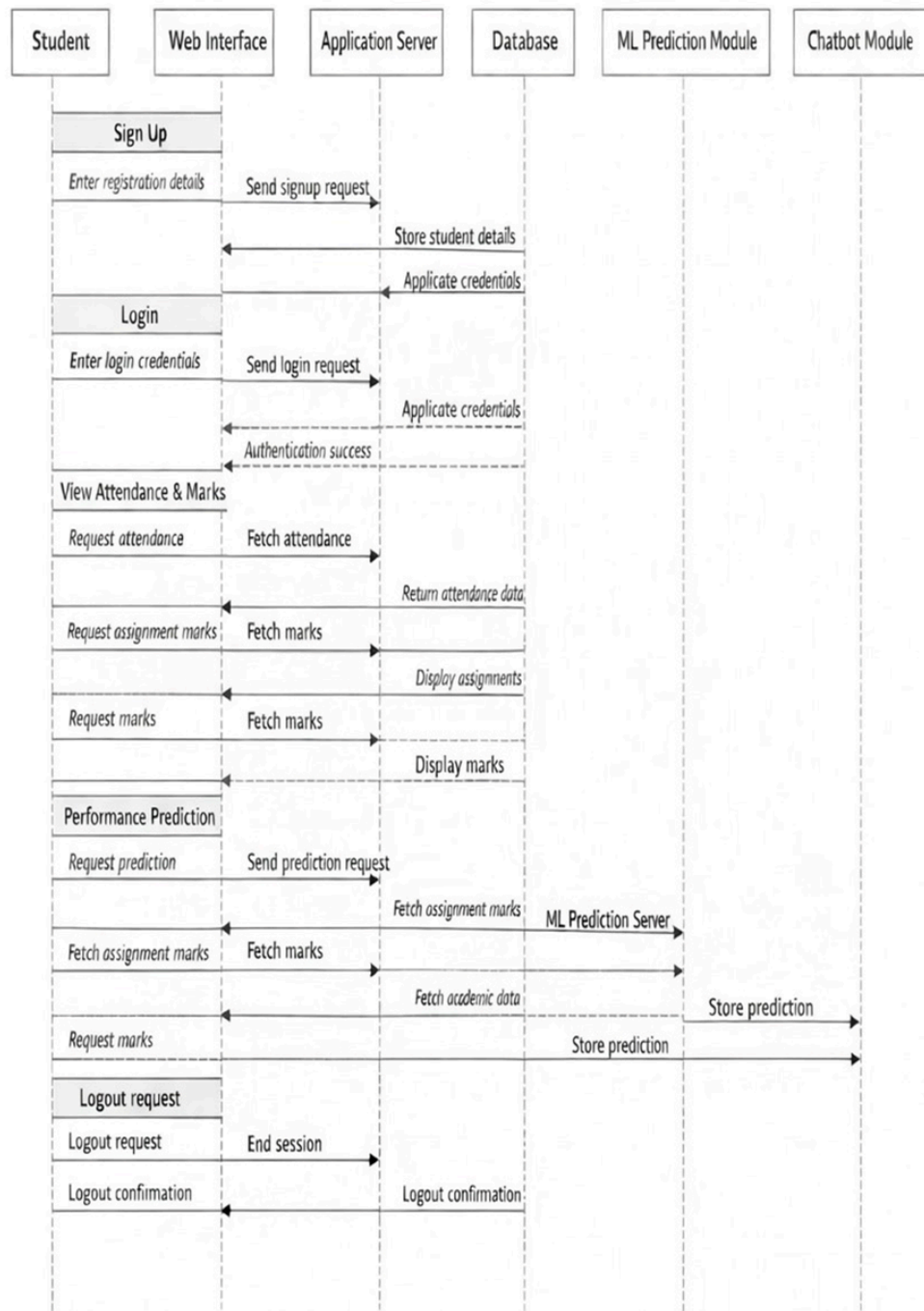
Admin Sequence Diagram – Description

The Admin Sequence Diagram illustrates the interaction between the Admin, Web Interface, Application Server, and Database during system management operations.



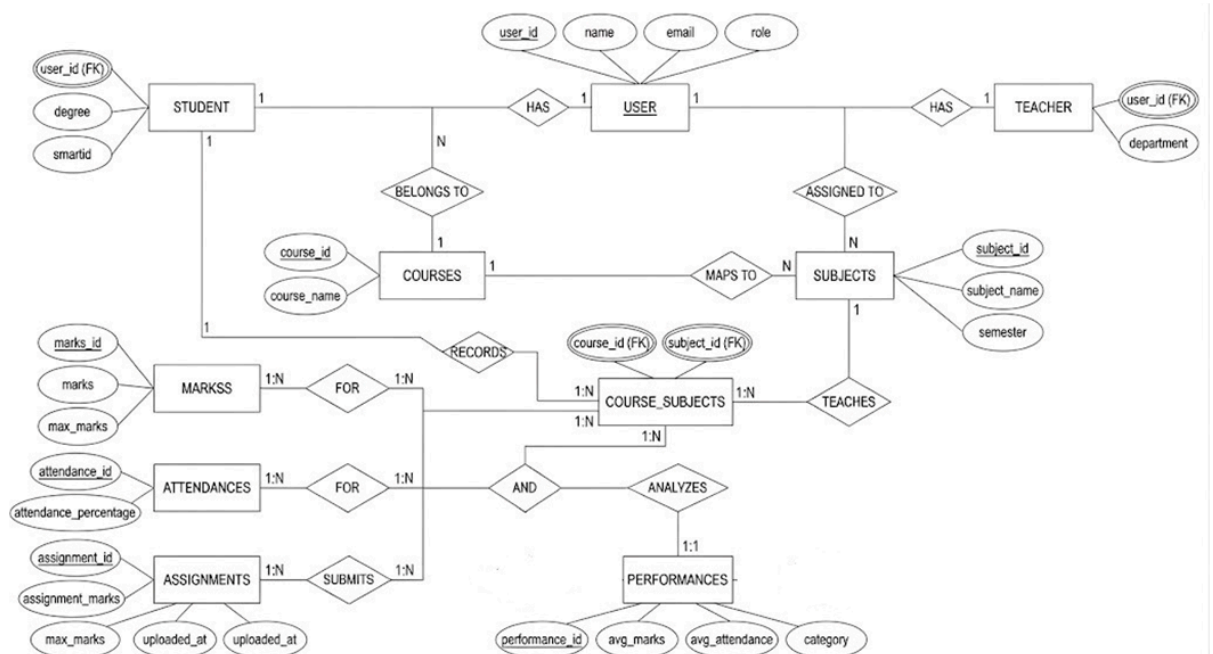
Student Sequence Diagram – Description

The Student Sequence Diagram represents the interaction between the Student, Web Interface, Application Server, Database, ML Prediction Module, and Chatbot Module.



2.2.4 E-R Diagram

The ER diagram represents a student performance prediction system where users are classified as students or teachers. Students have attendance and marks records linked to subjects. Teachers manage academic records. Based on attendance and marks, the system generates performance predictions for students.



3.TYPES OF TESTS (With Implementation)

Unit Testing

Unit testing is performed by developers during the development phase to verify the correctness of individual modules such as user registration, login, health ID generation, and consent management. These tests are executed on developers' local machines or a common development server. The unit test environment may differ from the target deployment environment; however, the transition to higher-level testing does not require significant code changes. One development machine per developer and a small shared server are sufficient for unit testing.

System Testing

System testing validates the complete and integrated Digital Health Management Record System by executing all modules together in real-world scenarios. This includes interaction between migrant workers, healthcare providers, and government authorities. If the deployment environment differs from the development environment, a system test machine with the same operating system and platform as the target environment is used. This environment ensures correct end-to-end functionality but remains moderately controlled and flexible.

Integration and Regression Testing

Integration and regression testing are carried out in a controlled environment similar to the target deployment setup. These tests ensure that newly integrated modules such as government scheme APIs or healthcare data uploads do not affect existing system functionality. A dedicated server is maintained for this purpose, and all changes are managed by the test coordinator to ensure stability, traceability, and reliable test results.

Stress Testing

Stress testing is conducted to evaluate system performance under high user load, such as simultaneous access by multiple healthcare providers and government users. This testing is performed in an environment identical to the production environment. Special attention is given to database performance, concurrency handling, and deadlock prevention, as these issues commonly arise under heavy load.

Acceptance Testing and Staging

Acceptance testing is carried out to verify that the system meets all functional and non-functional requirements specified by stakeholders. This phase is usually performed in the same environment used for stress testing. During initial deployment,

the production environment may be used; however, for future development cycles, a separate staging environment is maintained to validate updates before final release.

4. References

1. Python Software Foundation. *Python Documentation*. Python Software Foundation, <https://www.python.org/doc/>.
2. Django Software Foundation. *Django Official Documentation*. Django Software Foundation, <https://docs.djangoproject.com/>.
3. World Wide Web Consortium. *HTML Specification*. W3C, <https://html.spec.whatwg.org/>.
4. World Wide Web Consortium. *CSS Specification*. W3C, <https://www.w3.org/Style/CSS/>.
5. Mozilla Developer Network. *JavaScript Documentation*. MDN Web Docs, <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
6. Mozilla Developer Network. *HTML & CSS Documentation*. MDN Web Docs, <https://developer.mozilla.org/>.